



Ayudantía 6

INF129 Introducción a la Programación

Sergio Cárcamo Naranjo

September 22, 2026

DI - Universidad Técnica Federico Santa María

1. Funciones
2. Ejemplos
3. Recursividad
4. Problemas

Funciones

¿Qué es una función?

- Una **función** es un bloque de código reutilizable que realiza una tarea específica.
- Nos permite abstraer (ocultar detalles) y descomponer (dividir problemas grandes en más pequeños).
- Actúa como una “caja negra”: recibe entradas (parámetros), realiza cálculos, y entrega salidas (valor de retorno).

Definición y Llamado

Para definir una función en Python usamos la palabra clave `def`.

```
def saludar(nombre):  
    mensaje = "Hola " + nombre  
    return mensaje  
  
# Llamado a la función  
saludo = saludar("Mundo")  
print(saludo)
```

print() vs return

- `return`: Devuelve un valor que puede ser guardado en una variable o usado en otra expresión. La función termina su ejecución al retornar.
- `print`: Muestra texto en la pantalla, pero **no** devuelve ningún valor.

Alcance de Variables (Scope)

Las variables creadas **dentro** de una función solo existen dentro de ella (ámbito local). No pueden ser usadas desde afuera.

```
def mi_funcion():  
    variable_local = 10  
    print(variable_local)  
  
mi_funcion()  
  
# Esto tira error porque no existe fuera:  
# print(variable_local)
```

Orden de los Parámetros

Al llamar a una función, los argumentos se asignan a los parámetros en el **mismo orden** en que fueron definidos en su firma.

```
def restar(a, b):  
    return a - b  
  
# a = 10 y b = 3  
print(restar(10, 3)) # Imprime 7  
  
# El orden importa mucho:  
print(restar(3, 10)) # Imprime -7
```

Ejemplos

Ejemplo 1: Conversión de Temperaturas

Queremos una función que convierta grados Celsius a Fahrenheit.

```
def celsius_a_fahrenheit(c):  
    f = (c * 9/5) + 32  
    return f  
  
temp_c = 25  
temp_f = celsius_a_fahrenheit(temp_c)  
print(temp_c, "grados Celsius son", temp_f, "F°.")
```

Ejemplo 2: Número Primo

Una función que verifique si un número entero mayor a 1 es primo.

```
def es_primo(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True  
  
print("¿Es 7 primo?", es_primo(7))  
print("¿Es 10 primo?", es_primo(10))
```

Recursividad

¿Qué es la Recursividad?

- La **recursividad** es una técnica donde una función se llama **a sí misma** para resolver un problema.
- Todo problema recursivo debe tener:
 1. **Caso base:** Una condición de término simple que no requiere recursión (condición de parada).
 2. **Paso recursivo:** La función se llama a sí misma con un problema más pequeño, acercándose al caso base.

Ejemplo Recursivo: Factorial

El factorial de n se define como $n! = n \times (n - 1)!$, y $0! = 1$.

```
def factorial_recursivo(n):  
    # Caso base  
    if n == 0 or n == 1:  
        return 1  
    # Paso recursivo  
    else:  
        return n * factorial_recursivo(n - 1)  
  
print("El factorial de 5 es:", factorial_recursivo(5))
```

Ejemplo Recursivo: Fibonacci

Sucesión de Fibonacci: $F(0) = 0$, $F(1) = 1$, y $F(n) = F(n - 1) + F(n - 2)$.

```
def fibonacci(n):  
    # Casos base  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    # Paso recursivo  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
print("El término 6 de Fibonacci es:", fibonacci(6))
```

¿Dudas hasta aquí?

Problemas

Problema 1: Números Perfectos

Un número es **perfecto** si la suma de sus divisores propios (excluyéndose a sí mismo) es igual al número.

Escriba **dos** funciones para resolver este problema:

1. `suma_divisores(n)`: Que calcule y retorne la suma de los divisores propios de n .
2. `es_perfecto(n)`: Que reciba un número, utilice la función anterior y retorne `True` si es perfecto, o `False` en caso contrario.

Problema 2: MCD Recursivo

Ya conocemos el algoritmo de Euclides para calcular el Máximo Común Divisor (MCD) utilizando ciclos (`while`).

Escriba la función `mcd(a, b)` pero utilizando **recursividad**.

Recuerde la definición del algoritmo:

- **Caso base:** El MCD entre a y 0 es a .
- **Paso recursivo:** El MCD entre a y b es igual al MCD entre b y el resto de a dividido en b ($a \% b$).